



Step-by-Step Explanation: Relative Distance via Dot Product

You highlighted this:

Let's take PE(1) and PE(3), and dot them:

$$[0.8415, 0.5403, 0.0100, 0.99995] \cdot [0.1411, -0.9900, 0.0300, 0.99955]$$

$$= \mathbf{0.5837}$$

Now compute:

$$\cos(2) + \cos(0.02) = \mathbf{0.5837}$$

This shows: **dot product depends only on distance = 2**----- what is meant by dot product depends only on distance = 2 didnt understand

Let's break it into bite-sized steps:

+++++

Great — you've now selected this question:



What does "dot product depends only on distance = 2" mean?

You are looking at this expression:

$$\mathbf{PE(1) \cdot PE(3) = 0.5837}$$

Then calculating:

$$\mathbf{\cos(2) + \cos(0.02) = 0.5837}$$

So someone says:

"See? The dot product depends only on the **distance** between positions, which is $3 - 1 = 2$."

But what does this really mean?



What it doesn't mean

It does **not** mean that:

- PE(1) and PE(3) look the same
- Or that $\sin(1) = \sin(3)$
- Or that the values in the vectors repeat every 2 steps

No — it means something deeper, and mathematical.

✓ What it does mean

Let's take it slow:

The positional encoding is built as:

$$\mathbf{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{\{2i/d\}})$$

$$\mathbf{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{\{2i/d\}})$$

$$\text{So } \mathbf{PE}(\text{pos}) = [\sin(\omega_0 \cdot \text{pos}), \cos(\omega_0 \cdot \text{pos}), \sin(\omega_1 \cdot \text{pos}), \cos(\omega_1 \cdot \text{pos})]$$

Now, take two positions: $a = 1$, $b = 3$

Their **dot product**:

$$\mathbf{PE}(1) \cdot \mathbf{PE}(3)$$

$$= \sin(\omega_0 \cdot 1) \cdot \sin(\omega_0 \cdot 3) + \cos(\omega_0 \cdot 1) \cdot \cos(\omega_0 \cdot 3)$$

$$+ \sin(\omega_1 \cdot 1) \cdot \sin(\omega_1 \cdot 3) + \cos(\omega_1 \cdot 1) \cdot \cos(\omega_1 \cdot 3)$$

◆ Selected Equation to Explain:

$$\begin{aligned} \text{PE}(1) \cdot \text{PE}(3) &= \sin(\omega_0 \cdot 1) \cdot \sin(\omega_0 \cdot 3) + \cos(\omega_0 \cdot 1) \cdot \cos(\omega_0 \cdot 3) \\ \sin(\omega_1 \cdot 1) \cdot \sin(\omega_1 \cdot 3) &+ \cos(\omega_1 \cdot 1) \cdot \cos(\omega_1 \cdot 3) \end{aligned}$$

We'll now break this down and **derive it fully from scratch** so you understand where each term comes from.

🧱 Step 1: Background — How PE(pos) is built

We define:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Let's denote:

$$\omega_i = \frac{1}{10000^{2i/d}} \Rightarrow \theta_i(\text{pos}) = \omega_i \cdot \text{pos}$$

So:

$$\text{PE}(\text{pos}) = [\sin(\theta_0(\text{pos})), \cos(\theta_0(\text{pos})), \sin(\theta_1(\text{pos})), \cos(\theta_1(\text{pos}))]$$

🧠 Define:

$$\omega_i = \frac{1}{10000^{2i/d}} \quad (\text{this is just a substitution})$$

This is the **frequency** used in the sin/cos calculation at dimension pair i .

Then the argument of sin and cos becomes:

$$\frac{\text{pos}}{10000^{2i/d}} = \text{pos} \cdot \frac{1}{10000^{2i/d}} = \text{pos} \cdot \omega_i$$

Now we define:

$$\theta_i(\text{pos}) := \omega_i \cdot \text{pos}$$

That is:

$\theta_i(\text{pos})$ is the **angle** used in the sine or cosine function for position pos and dimension index i .

12
34**Step 2: Write out PE(1) and PE(3)**

Let's define:

- $\theta_0(1) = \omega_0 \cdot 1$
- $\theta_0(3) = \omega_0 \cdot 3$
- $\theta_1(1) = \omega_1 \cdot 1$
- $\theta_1(3) = \omega_1 \cdot 3$

Then:

- $PE(1) = [\sin(\theta_0(1)), \cos(\theta_0(1)), \sin(\theta_1(1)), \cos(\theta_1(1))]$
- $PE(3) = [\sin(\theta_0(3)), \cos(\theta_0(3)), \sin(\theta_1(3)), \cos(\theta_1(3))]$

**Step 3: Compute the dot product**

We now do a **dot product** of PE(1) and PE(3), i.e.:

$$PE(1) \cdot PE(3) = \sin(\theta_0(1)) \cdot \sin(\theta_0(3)) + \cos(\theta_0(1)) \cdot \cos(\theta_0(3)) \\ \sin(\theta_1(1)) \cdot \sin(\theta_1(3)) + \cos(\theta_1(1)) \cdot \cos(\theta_1(3))$$



This is exactly what you highlighted:

$$\sin(\omega_0 \cdot 1) \cdot \sin(\omega_0 \cdot 3) + \cos(\omega_0 \cdot 1) \cdot \cos(\omega_0 \cdot 3) \\ \sin(\omega_1 \cdot 1) \cdot \sin(\omega_1 \cdot 3) + \cos(\omega_1 \cdot 1) \cdot \cos(\omega_1 \cdot 3)$$

**Step 4: Apply the trigonometric identity**

Recall the identity:

$$\sin(a) \sin(b) + \cos(a) \cos(b) = \cos(b - a)$$

We apply this separately for each frequency:

- For ω_0 :

$$\cos(\omega_0 \cdot (3 - 1)) = \cos(2\omega_0)$$

- For ω_1 :

$$\cos(\omega_1 \cdot (3 - 1)) = \cos(2\omega_1)$$

So the dot product becomes:

$$PE(1) \cdot PE(3) = \cos(2\omega_0) + \cos(2\omega_1)$$



✓ Final Interpretation

You just derived that:

The dot product between PE(1) and PE(3) =
 $\cos(\omega_0 \cdot (3 - 1)) + \cos(\omega_1 \cdot (3 - 1))$
= A function of **distance = 2** — not the absolute positions themselves

This proves the claim:

"Dot product depends only on the distance between positions"

Let me know if you'd like to try this out numerically (e.g. with $\omega_0 = 1$, $\omega_1 = 1/100$) — and compute the exact result for fun!

✓ Final Interpretation

You just derived that:

The dot product between PE(1) and PE(3) =
 $\cos(\omega_0 \cdot (3 - 1)) + \cos(\omega_1 \cdot (3 - 1))$
= A function of **distance = 2** — not the absolute positions themselves

This proves the claim:

"Dot product depends only on the distance between positions"

Now apply the identity:

$$\sin(a) \cdot \sin(b) + \cos(a) \cdot \cos(b) = \cos(b - a)$$

So:

$$= \cos(\omega_0 \cdot (3-1)) + \cos(\omega_1 \cdot (3-1)) = \cos(2) + \cos(0.02) = 0.5837$$

And **that's the point**:

✓ The dot product depends only on $\Delta = (\text{pos2} - \text{pos1})$

In this case:

$$\Delta = 3 - 1 = 2$$

That means:

- It doesn't matter if you're comparing PE(1) and PE(3), or PE(20) and PE(22), or PE(100) and PE(102)
- As long as the **gap = 2**, the dot product will be **the same**

🎯 This is what "depends only on distance = 2" means!

🔄 Why is this important?

This means the Transformer can **sense** that two tokens are 2 steps apart **no matter where they are in the sentence** — because the math makes that relationship pop out directly via the dot product.

Demonstrate:

1 Compute Frequencies (ω_i):

- For $i = 0$:

$$\omega_0 = 10000^{-2 \times 0/4} = 1$$

- For $i = 1$:

$$\omega_1 = 10000^{-2 \times 1/4} = 10000^{-0.5} = 0.01$$

2 Compute PE vectors explicitly:

$$\bullet \text{ } PE(1) = [\sin(1), \cos(1), \sin(0.01), \cos(0.01)]:$$

$$[0.84147, 0.54030, 0.0099998, 0.99995]$$

$$\bullet \text{ } PE(3) = [\sin(3), \cos(3), \sin(0.03), \cos(0.03)]:$$

$$[0.14112, -0.98999, 0.0299955, 0.99955]$$

3 Dot product explicitly calculated:

$$PE(1) \cdot PE(3) = (0.84147)(0.14112) + (0.54030)(-0.98999) + (0.0099998)(0.0299955) + (0.99995)(0.99955)$$

Calculate step by step:

- $0.84147 \times 0.14112 = 0.1188$
- $0.54030 \times (-0.98999) = -0.5349$
- $0.0099998 \times 0.0299955 = 0.0003$
- $0.99995 \times 0.99955 = 0.9995$

Summation:

$$0.1188 - 0.5349 + 0.0003 + 0.9995 = 0.5837$$

4 Using the compact cosine form directly (from the selected formula):

The selected statement says the dot product can also be computed as:

$$\sum_i \cos(\omega_i \cdot (pos - pos)) = \cos(\omega_0 \times 2) + \cos(\omega_1 \times 2)$$

Plugging in values:

- $\cos(\omega_0 \times 2) = \cos(2) = -0.4161$
- $\cos(\omega_1 \times 2) = \cos(0.02) = 0.9998$

Sum:

$$-0.4161 + 0.9998 = 0.5837$$

✓ Observations:

- Both methods yield exactly the same result: **0.5837**.
- The dot product explicitly depends only on the positional **distance ($\Delta=2$)**, not individual positions 1 or 3.

1 Dot Product Reflects Positional Relationship

- The transformer computes the dot product between positional encodings $PE(pos_1) \cdot PE(pos_2)$.
- Due to trigonometric identities, this dot product **depends only on the distance between positions**, not on the absolute values.

So:

- $PE(1) \cdot PE(3) = PE(5) \cdot PE(7) = PE(10) \cdot PE(12)$
- All these pairs are **2 steps apart** → same dot product → **same relative position signal**.

2 Self-Attention Sees: "This Token is 2 Steps Away"

In the attention mechanism:

- Queries and keys are positionally encoded.
- The dot product between these encodings affects how much **attention** is paid between token pairs.

So, if a token at position t "looks" at another token $t + 2$, it will **see the same encoding signal** no matter where in the sentence they are.

✓ Hence, the transformer **learns that a token is 2 steps away**, regardless of whether that means from 1 to 3, 5 to 7, or 100 to 102.

Can you demonstrate the above i.e. $PE(1) \cdot PE(3) = PE(5) \cdot PE(7) = PE(10) \cdot PE(12)$ by giving another example to demonstrate- show the vectorized calculations as before for that

✓ When d is Even

When d is even:

- We have $d/2$ full sin-cos pairs.

- For each pair, we can apply:

$$\sin(\omega_i pos_1) \cdot \sin(\omega_i pos_2) + \cos(\omega_i pos_1) \cdot \cos(\omega_i pos_2) = \cos(\omega_i (pos_2 - pos_1))$$

- This lets us derive the clean formula:

$$PE(pos_1) \cdot PE(pos_2) = \sum_{i=0}^{d/2-1} \cos(\omega_i \cdot \Delta)$$

✓ This is exact and provable.

! When d is Odd

Now, suppose $d = 5$:

- We only have $\lfloor 5/2 \rfloor = 2$ full sin-cos pairs.
- And 1 extra unpaired sine term (dimension 4, $i = 2$).

So your PE vector looks like:

$$PE(pos) = [\sin(\omega_0 pos), \cos(\omega_0 pos), \sin(\omega_1 pos), \cos(\omega_1 pos), \sin(\omega_2 pos)]$$

! Problem:

For $i = 2$, there's **only a sine**, no cosine.

So we **can't apply the identity**:

$$\sin a \cdot \sin b + \cos a \cdot \cos b = \cos(b - a)$$

Because we **don't have** the cosine term $\cos(\omega_2 pos)$.

So What About:

$$PE(3) \cdot PE(5) = PE(5) \cdot PE(7) = PE(10) \cdot PE(12)$$

This equality is **approximately true**, but **not exact** when d is odd.

Why?

- For all those pairs, $\Delta = 2$, so the cosine-based part is identical.
- But the **extra sine-only term** adds a **different small value** to each dot product:
 $\sin(\omega_2 \cdot 3) \cdot \sin(\omega_2 \cdot 5) \neq \sin(\omega_2 \cdot 5) \cdot \sin(\omega_2 \cdot 7)$

Hence:

-  The bulk of the dot product still **tracks distance**
-  But there's a **small dependency on absolute positions**

Hence:

-  The bulk of the dot product still **tracks distance**
-  But there's a **small dependency on absolute positions**

Conclusion:

- For **even** d :

$$PE(pos_1) \cdot PE(pos_2) = \sum_i \cos(\omega_i \cdot \Delta) \Rightarrow \text{Purely distance -- dependent}$$

- For **odd** d :

$$PE(pos_1) \cdot PE(pos_2) = \sum_i \cos(\omega_i \cdot \Delta) + \underbrace{\sin(\omega_r \cdot pos_1) \cdot \sin(\omega_r \cdot pos_2)}_{\text{position-dependent correction}}$$

So the **equality across different position pairs with same distance** is **approximately true**, not mathematically exact.

query on semantic +positional awareness combined

3 Semantic + Positional Awareness Combined

- Word embeddings tell the transformer **what** a word is (e.g., "cat" vs. "sat").
- Positional encodings tell it **where** it is in the sentence — more precisely, **how far** one word is from another.

Together, this lets attention say:

"I should pay attention to that word 2 tokens before me — and that's true whether I'm at the beginning, middle, or end of the sentence."

-mixing of word embedding with positional encoding- do the positional awareness as mentioned in the previous example remain the same after the mixing? Can you demonstrate by modifying the given example above??

Short Answer

❌ No — after mixing PE with word embeddings, the dot product is no longer purely distance-based.

But:

✅ The **positional information is still present** — it's just now entangled with semantic (word) information.

Why the Pattern Breaks

Before mixing:

- $PE(pos_1) \cdot PE(pos_2) = \sum \cos(\omega_i \cdot (pos_2 - pos_1))$
- So **same $\Delta \rightarrow$ same dot product** (clean positional awareness)

After mixing:

$$X_1 = WE_1 + PE_1, \quad X_3 = WE_3 + PE_3$$

$$X_1 \cdot X_3 = WE_1 \cdot WE_3 + WE_1 \cdot PE_3 + PE_1 \cdot WE_3 + PE_1 \cdot PE_3$$

Only the **last term** reflects clean positional distance.

All other terms bring in **word-specific variation**.



Question #12 -

✅ **Positional signal exists**, but ❌ **not invariant** to relative distance anymore- then how does PE helps in positional awareness once mixed with WE??

Absolutely — here is a focused and complete response to **Question #12** from your notes:

? "✅ Positional signal exists, but ❌ not invariant to relative distance anymore — then how does PE help in positional awareness once mixed with WE?"

✅ Great question. Here's the crux:

You're asking:

If the **clean distance-aware property** (like $PE(1) \cdot PE(3) = PE(5) \cdot PE(7)$) is **no longer preserved** after mixing positional encoding (PE) with word embedding (WE), then...

How does the transformer still retain positional awareness?

Let's break this down with precision.

🔄 **Step-by-Step Clarification**

◆ 1. What was happening before mixing

Before adding PE and WE:

$$\bullet \text{ } PE(pos) \cdot PE(pos + \Delta)$$

depends only on Δ , **not** on absolute positions.

- So positional relationships like "2 steps apart" were **perfectly encoded**.

◆ 2. What changes after mixing

After mixing:

$$X(pos) = WE(pos) + PE(pos)$$

Then:

$$X(pos_1) \cdot X(pos_2) = WE_1 \cdot WE_2 + PE_1 \cdot PE_2 + \text{cross terms}$$

Only $PE_1 \cdot PE_2$ is distance-sensitive.

But the dot product now includes:

- Terms based on **word content**
- And **word–position interactions**

But the dot product now includes:

- Terms based on **word content**
- And **word–position interactions**

So:

✗ You can't isolate distance just by looking at the final dot product anymore

🎯 So Then — How Does Positional Awareness Survive?

Here's the key insight:

✅ 3. The position signal is still embedded in the vector

Even though:

- The **positional contribution is now entangled** with word semantics,
- The vector still contains **both signals** — they're just **not separated**

linearly anymore

Transformers have:

- **Multi-head attention**
- **Learnable weights** in **Q**, **K**, and **V** matrices

This means:

The model learns to **extract and use the positional signal**, even when it's buried inside a mixed vector.

Each attention head might learn:

- One to focus more on **semantic similarity**
- Another to focus on **relative or absolute positions**

So instead of relying on dot product = $\cos(\Delta)$, the transformer learns to **decode** that signal internally.

✅ Think of It This Way

- The raw sinusoidal PE is like a "GPS coordinate" of where a word sits.
- Adding it to a word embedding is like putting the GPS tag on a word.
- The transformer doesn't compare GPS tags directly via dot products anymore — but it **learns how to read them** through attention.

So:

🔥 The relative position signal is **still there** — just not immediately visible in a dot product.

Final Answer

Even after mixing PE with WE, the transformer retains positional awareness because:

- The PE values are still **part of the input vector**
- The model **learns to attend to positional cues** using its internal Q/K projections
- It can thus still figure out “who’s nearby” and “who’s far” — even though that’s no longer visible via simple dot products.

Would you like a visualization showing how different attention heads behave with and without positional encodings?

=====

=====

=====

Sinusoidal-Positional-Encoding...

PDF Document · 1.5 MB

